Technical Project Explanation

This project is a full-stack AI interview preparation system built to help students and job seekers practice interviews in a personalized and realistic way. It combines resume analysis, question generation, and AI-powered feedback to simulate a guided interview experience.

1. Project Overview

The system addresses common weaknesses in preparation: generic questions, lack of role relevance, and limited real-time feedback. Users can either upload a resume or choose a target role. The application then generates tailored interview questions and evaluates their responses using Google Gemini AI.

2. Core Functionalities

- Resume-based mode: upload a PDF resume and extract technical skills, projects, and experience trends.
- Role-based mode: generate industry-specific questions from a selected job role and difficulty level.
- Interview session tracking: save questions, answers, and user performance state in the database.
- AI-driven feedback: evaluate response quality, strengths, and improvement areas after interview completion.

3. System Architecture

The application follows a layered architecture. The frontend is a React application built with Vite. The backend is a Flask service that handles route logic, auth, file handling, and AI orchestration. The database uses SQLAlchemy models, while AI features are powered by Google Gemini. Resume parsing is handled through pdfplumber and fallback support from PyPDF2.

4. Technology Stack

- Frontend: React, Vite, React Router, custom CSS.
- Backend: Python, Flask, Flask-SQLAlchemy, Flask-Login, Flask-CORS.
- AI: Google Gemini API for question generation and response evaluation.
- Authentication: Google OAuth 2.0 using flask-login and oauthlib patterns.
- Data / file processing: SQLite for local storage and PDF parsing with pdfplumber.

5. Key Backend Modules

The backend entry point is app.py, which configures the Flask app and exposes endpoints such as /api/upload-resume, /api/generate-questions, /api/submit-answer, and /api/complete-interview. Models are defined in models.py, where User and InterviewSession encapsulate user identity and session lifecycle. google_auth.py implements the Google OAuth flow, while resume_analyzer.py extracts skills and project context from uploaded PDF resumes. The question generation and feedback logic rely on Gemini-based AI orchestration.

6. Frontend Experience

The frontend is a modern single-page application with routing for the login screen, dashboard, resume upload, interview flow, and feedback pages. It manages authentication state and interacts with the backend through API requests, creating a clean and responsive user experience.

7. Data Flow

- User signs in using Google OAuth.
- User uploads a resume or selects a role.
- AI analyzes resume content or role context to generate questions.
- User answers the questions during the session.
- The system stores answers and produces feedback at the end of the interview.

8. Strengths and Extensibility

This project demonstrates an effective use of LLMs in practical interview preparation. It is modular, scalable, and suitable for extension with richer analytics, additional roles, better deployment configuration, and more robust evaluation scoring. The architecture clearly separates concerns between frontend interaction, API logic, data persistence, and AI processing.

Conclusion

The LLM-Powered Cognitive Interview Assistant is a practical and modern solution for interview preparation. By combining resume-based personalization with AI-generated questions and actionable feedback, it offers a powerful platform for skill development and interview readiness.